Universidad Católica Boliviana

INFORME TÉCNICO

Herbario de Flora Boliviana

Arquitectura orientada a servicios: API Gateway (proxy inverso) + Microservicio NestJS + PostgreSQL, orquestados con Docker Compose

Estudiante	[Nombre completo del estudiante]
Materia	[Nombre de la materia]
Docente	[Nombre del docente]
Tecnología asignada	NestJS (TypeScript)
Base de datos	PostgreSQL 16
Repositorio	[URL del repositorio público en GitHub/GitLab]
Fecha	Octubre de 2026

1. Descripción del dominio

1.1 Problema de negocio

Bolivia es un país megadiverso, con ecorregiones muy distintas entre sí: Altiplano, valles, Yungas, Chaco, Amazonía y Llanos. Sin embargo, la información sobre su flora suele estar dispersa en planillas, cuadernos de campo e informes aislados. Esto dificulta saber *qué especies existen*, *a qué familia botánica pertenecen*, *en qué rango de altitud viven*, *cuál es su estado de conservación* y, sobre todo, *dónde y cuándo fue observada* cada una, de modo que no hay trazabilidad ni una fuente única y consultable.

1.2 Solución propuesta

Se construye un **herbario digital** expuesto como API REST. El sistema centraliza el catálogo de plantas con su familia, ecorregión, rango altitudinal y estado de conservación, y permite registrar **avistamientos** georreferenciados (localidad, departamento, coordenadas, fecha y observador). Cualquier cliente (una app web o móvil, Postman, un script de investigación) consume el servicio a través de un único punto de entrada: el API Gateway, que autentica y controla el tráfico antes de llegar al microservicio.

1.3 Actores

Actor	Rol en el sistema
Investigador botánico	Registra y actualiza plantas, familias y avistamientos de campo.
Cliente de la API	Aplicación o persona que consulta el catálogo (búsquedas y filtros).
Administrador	Posee las credenciales del gateway y gestiona la configuración del despliegue.

1.4 Entidades del dominio

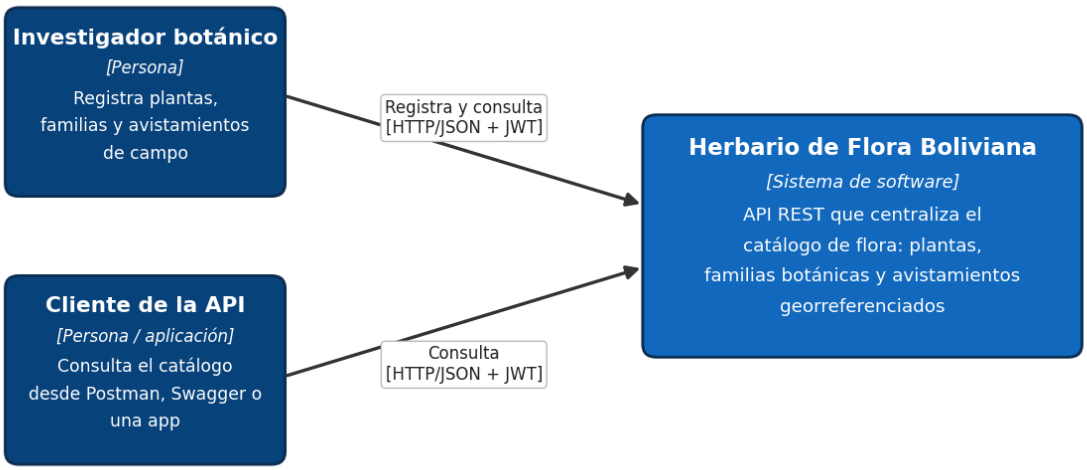
Entidad	Atributos principales	Relaciones
Familia	id, nombre (único), descripción	Una familia agrupa muchas plantas (1:N).
Planta	id, nombre científico (único), nombre común, descripción, ecorregión, altitud mín./máx., estado de conservación	Pertenece a una familia (N:1). Tiene muchos avistamientos (1:N).
Avistamiento	id, localidad, departamento, latitud, longitud, fecha, observador, notas	Pertenece a una planta (N:1).

1.5 Reglas de negocio

• El nombre científico de la planta y el nombre de la familia son únicos (conflicto 409 si se repiten).
• Toda planta debe pertenecer a una familia existente.
• No se puede eliminar una familia que todavía tiene plantas asociadas (restricción *RESTRICT*, 409).
• Al eliminar una planta se eliminan sus avistamientos (*CASCADE*).
• La altitud mínima no puede ser mayor que la máxima (400).
• El estado de conservación pertenece a un conjunto cerrado de categorías, inspirado en la Lista Roja de la UICN: preocupación menor, casi amenazada, vulnerable, en peligro y en peligro crítico.
• El departamento del avistamiento debe ser uno de los nueve departamentos de Bolivia y la fecha usa el formato AAAA-MM-DD.

2. Arquitectura del sistema (Modelo C4)

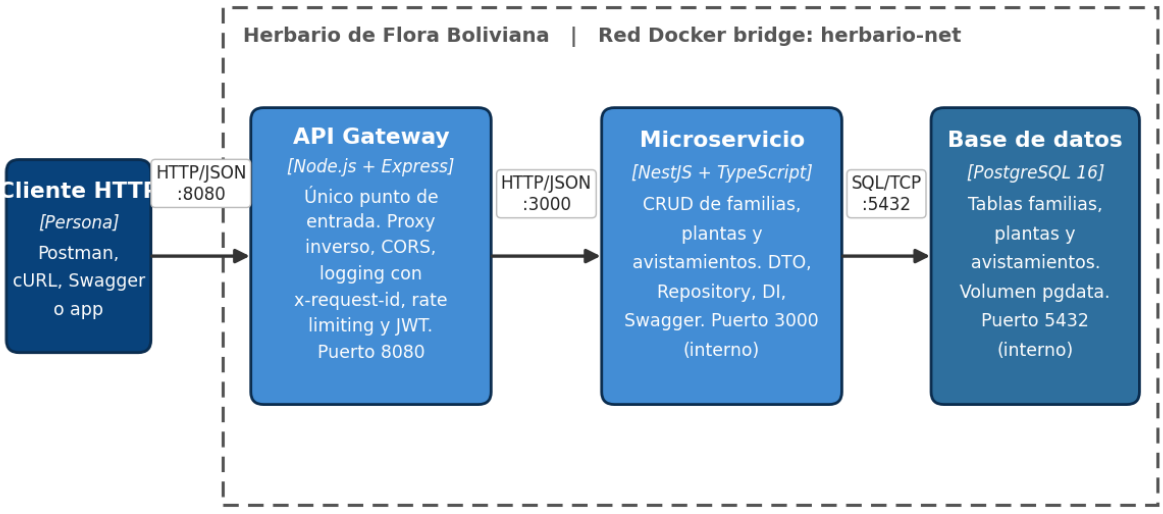
2.1 Nivel 1 — Diagrama de Contexto



C4 - Nivel 1: Diagrama de Contexto del Sistema

El sistema **Herbario de Flora Boliviana** es consumido por dos tipos de actores humanos o aplicaciones, siempre por HTTP/JSON y con un token JWT. No depende de sistemas externos: toda la información vive en su propia base de datos.

2.2 Nivel 2 — Diagrama de Contenedores



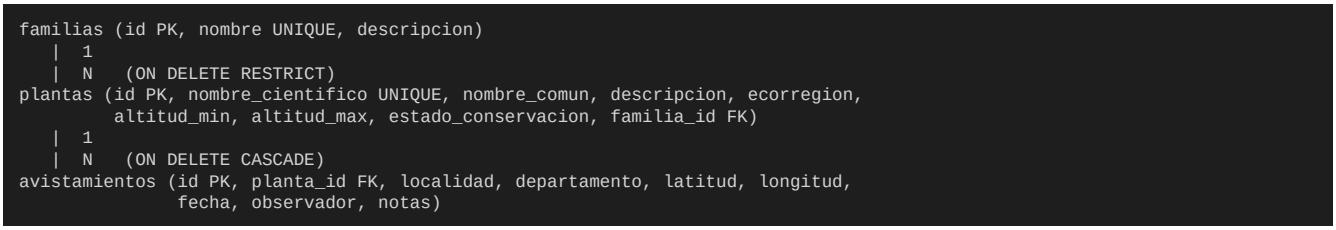
C4 - Nivel 2: Diagrama de Contenedores

Contenedor	Tecnología	Responsabilidad	Exposición
API Gateway	Node.js 20, Express, http-proxy-middleware	Punto único de entrada; enruta /api y /docs; CORS, logging con x-request-id, rate limiting, JWT.	Puerto 8080 publicado al host
Microservicio	NestJS 10, TypeScript, TypeORM, class-validator, Swagger	Lógica de negocio y CRUD de familias, plantas y avistamientos (DTO, Repository, DI).	Puerto 3000, solo red interna
Base de datos	PostgreSQL 16 (imagen alpine)	Persistencia relacional con volumen `pgdata`.	Puerto 5432, solo red interna

2.3 Decisiones de diseño

• **Gateway como única puerta:** solo el puerto 8080 se publica. El microservicio y PostgreSQL no son accesibles desde fuera de la red `herbario-net` (driver *bridge*).
• **Características transversales en el gateway:** CORS configurable, logging JSON con `x-request-id` propagado al microservicio (trazabilidad de extremo a extremo), rate limiting (100 peticiones/min por IP y 10 intentos/min en el login) y validación de JWT en todas las rutas `/api`.
• **Patrones en el microservicio:** *Repository* (servicios que inyectan `Repository<Entidad>` de TypeORM), *DTO* con validación (whitelist y forbidNonWhitelisted) e *inyección de dependencias* de NestJS.
• **Contenerización:** Dockerfile multi-stage para el microservicio (compila y luego copia solo lo necesario, ejecuta como usuario no root) y Dockerfile ligero para el gateway.
• **Orquestación:** `docker-compose.yml` con `depends_on` en condición `service_healthy` y healthchecks en los tres servicios: la base de datos arranca primero, luego el microservicio y al final el gateway.
• **Manejo de errores:** las violaciones de unicidad y de llaves foráneas de PostgreSQL se traducen a respuestas 409; si el microservicio cae, el gateway responde 502 en lugar de quedarse colgado.

2.4 Modelo de datos



3. Documentación de endpoints

Todas las rutas se consumen a través del gateway: `http://localhost:8080`. Las rutas `/api/*` exigen el encabezado `Authorization: Bearer <token>`. La documentación interactiva (Swagger UI) está en `/docs`.

Método	URL	Descripción	Respuestas
POST	/auth/login	Autentica y devuelve el JWT	200, 401, 429
GET	/health	Estado del gateway	200
GET	/api/familias	Lista las familias	200
GET	/api/familias/:id	Obtiene una familia	200, 404
POST	/api/familias	Crea una familia	201, 400, 409
PATCH	/api/familias/:id	Actualiza una familia	200, 400, 404, 409
DELETE	/api/familias/:id	Elimina una familia (sin plantas)	204, 404, 409
GET	/api/plantas?familiaId&ecorregion;&estado;&q;	Lista plantas con filtros y búsqueda	200, 400
GET	/api/plantas/:id	Obtiene una planta (incluye su familia)	200, 404
GET	/api/plantas/:id/avistamientos	Avistamientos de una planta	200, 404
POST	/api/plantas	Crea una planta	201, 400, 409
PATCH	/api/plantas/:id	Actualiza una planta	200, 400, 404, 409
DELETE	/api/plantas/:id	Elimina una planta y sus avistamientos	204, 404
GET	/api/avistamientos?plantaId&departamento;	Lista avistamientos con filtros	200, 400
GET	/api/avistamientos/:id	Obtiene un avistamiento	200, 404
POST	/api/avistamientos	Registra un avistamiento	201, 400, 409
PATCH	/api/avistamientos/:id	Actualiza un avistamiento	200, 400, 404, 409
DELETE	/api/avistamientos/:id	Elimina un avistamiento	204, 404

Cuando falta el token o es inválido el gateway responde 401; si se excede el límite de peticiones, 429; y si el microservicio no responde, 502.

3.1 Estructuras JSON

Login — `POST /auth/login`

```
Solicitud:  {"username": "admin", "password": "herbario123"}
Respuesta 200: {"access_token": "<jwt>", "token_type": "Bearer", "expires_in": "1h"}
```

Crear planta — `POST /api/plantas`

```
Solicitud:
{ "nombreCientifico": "Echinopsis tarijensis", "nombreComun": "Cardón",
  "ecorregion": "Valles secos", "altitudMin": 1800, "altitudMax": 3200,
  "estadoConservacion": "preocupacion_menor", "familiaId": 6 }
Respuesta 201:
{ "id": 6, "nombreCientifico": "Echinopsis tarijensis", "nombreComun": "Cardón",
  "ecorregion": "Valles secos", "altitudMin": 1800, "altitudMax": 3200,
  "estadoConservacion": "preocupacion_menor", "familiaId": 6,
  "familia": { "id": 6, "nombre": "Cactaceae", "descripcion": "Familia de los cactus" } }
```

Registrar avistamiento — `POST /api/avistamientos`

```
Solicitud:
{ "plantaId": 6, "localidad": "Tarija - Tolomosa", "departamento": "Tarija",
  "latitud": -21.5, "longitud": -64.8, "fecha": "2026-10-10",
  "observador": "Estudiante UCB", "notas": "Ejemplar adulto" }
Respuesta 201: el avistamiento creado, con "id" y la "planta" asociada.
```

Errores

```
400: {"message": ["nombreCientifico should not be empty", ...], "error": "Bad Request", "statusCode": 400}
401: {"error": "Unauthorized", "message": "Falta el token Bearer."}
409: {"message": "Ya existe un registro con esos datos únicos.", "error": "Conflict", "statusCode": 409}
502: {"error": "Bad Gateway", "message": "El microservicio no responde."}
```

4. Evidencia de pruebas

Las pruebas se ejecutan con el script `scripts/smoke-test.sh` (20 casos de punta a punta contra el gateway) y con la colección de Postman incluida en el repositorio. Las secciones 4.1 a 4.4 muestran la salida real de esa ejecución sobre el gateway, el microservicio y PostgreSQL 16 (procesos locales durante el desarrollo); la sección 4.5 reúne las capturas de la ejecución con Docker Compose.

4.1 Resultados de la batería de pruebas

#	Caso de prueba	Petición	Esperado	Obtenido
1	Health del gateway	GET /health	200	200 OK
2	Acceso SIN token	GET /api/plantas	401	401 OK
3	Login con credenciales incorrectas	POST /auth/login	401	401 OK
4	Login correcto	POST /auth/login	200	200 OK
5	Listar familias	GET /api/familias	200	200 OK
6	Crear familia	POST /api/familias	201	201 OK
7	Crear familia duplicada	POST /api/familias	409	409 OK
8	Crear planta	POST /api/plantas	201	201 OK
9	Crear planta con datos inválidos	POST /api/plantas	400	400 OK
10	Buscar plantas por nombre	GET /api/plantas?q=card%C3%B3n	200	200 OK
11	Filtrar plantas en peligro	GET /api/plantas?estado=en_peligro	200	200 OK
12	Obtener planta por id	GET /api/plantas/8	200	200 OK
13	Actualizar planta	PATCH /api/plantas/8	200	200 OK
14	Registrar avistamiento	POST /api/avistamientos	201	201 OK
15	Avistamientos de la planta	GET /api/plantas/8/avistamientos	200	200 OK
16	Borrar familia con plantas	DELETE /api/familias/12	409	409 OK
17	Borrar avistamiento	DELETE /api/avistamientos/4	204	204 OK
18	Borrar planta	DELETE /api/plantas/8	204	204 OK
19	Borrar familia	DELETE /api/familias/12	204	204 OK
20	Planta borrada ya no existe	GET /api/plantas/8	404	404 OK

4.2 Ejemplos de salida

```
$ curl -s -X GET http://localhost:8080/api/plantas
{"error":"Unauthorized","message":"Falta el token Bearer."}
HTTP 401

$ curl -s -X POST http://localhost:8080/auth/login -d '{"username":"admin","password":"herbario123"}'
{"access_token":"eyJhbGciOiJIUzI1NiIs...(recortado)","token_type":"Bearer","expires_in":"1h"}
HTTP 200

$ curl -s -X POST http://localhost:8080/api/plantas -d '{"nombreCientifico":"Echinopsis
    tarijensis","nombreComun":"Cardón","ecorregion":"Valles
    secos","altitudMin":1800,"altitudMax":3200,"familiaId":12}'
{"id":8,"nombreCientifico":"Echinopsis tarijensis","nombreComun":"Cardón","descripcion":null,"ecorregion":"Valles seco
    s","altitudMin":1800,"altitudMax":3200,"estadoConservacion":"preocupacion_menor","familiaId":12,"familia":{"id":12
    ,"nombre":"Cactaceae","descripcion":"Familia de los cactus"}}
HTTP 201

$ curl -s -X POST http://localhost:8080/api/familias -d '{"nombre":"Cactaceae"}'
{"message":"Ya existe un registro con esos datos únicos.","error":"Conflict","statusCode":409}
HTTP 409

$ curl -s -X DELETE http://localhost:8080/api/familias/12
{"message":"Operación inválida por relaciones: el registro referenciado no existe o aún tiene registros
    asociados.","error":"Conflict","statusCode":409}
HTTP 409
```

4.3 Trazabilidad: el mismo x-request-id en gateway y microservicio

```
[gateway]       {"time":"2026-10-18T17:04:03.068Z","requestId":"8c99f0be-be61-47fb-8b8c-
    0f60a74f161c","method":"POST","url":"/api/plantas","status":201,"ms":15.542,"user":"admin","ip":"127.0.0.1"}
[microservicio] [HTTP] [8c99f0be-be61-47fb-8b8c-0f60a74f161c] POST /api/plantas 201 16ms

[gateway]       {"time":"2026-10-18T17:04:03.238Z","requestId":"12ab89aa-1ac5-49b1-92c8-
    ac86c0a9829a","method":"DELETE","url":"/api/familias/12","status":409,"ms":10.137,"user":"admin","ip":"127.0.0.1"}
[microservicio] [HTTP] [12ab89aa-1ac5-49b1-92c8-ac86c0a9829a] DELETE /api/familias/12 409 8ms
```

4.4 Rate limiting y error 502

```
$ for i in $(seq 1 12); do curl -s -o /dev/null -w "intento $i -> HTTP %{http_code}
" -X POST http://localhost:8080/auth/login -H "Content-Type: application/json" -d "{"username":"x","password":"x"}";
    done
intento 1 -> HTTP 401
intento 2 -> HTTP 401
intento 3 -> HTTP 401
intento 4 -> HTTP 401
intento 5 -> HTTP 401
intento 6 -> HTTP 401
intento 7 -> HTTP 401
intento 8 -> HTTP 401
intento 9 -> HTTP 401
intento 10 -> HTTP 401
intento 11 -> HTTP 429
intento 12 -> HTTP 429

$ curl -i -X POST http://localhost:8080/auth/login ...   (respuesta cuando se excede el límite)
HTTP/1.1 429 Too Many Requests
RateLimit-Limit: 10
RateLimit-Remaining: 0
{"error":"Too Many Requests","message":"Demasiados intentos de login, intenta de nuevo en un momento."}
```

```
# docker-compose stop microservicio   (simulado: se detiene el proceso del microservicio)
$ curl -i http://localhost:8080/api/plantas -H "Authorization: Bearer <token>"
HTTP/1.1 502 Bad Gateway
{"error":"Bad Gateway","message":"El microservicio no responde."}

# Log del gateway en ese momento:
{"level":"error","requestId":"b8ea826b-21ea-45ac-8433-d989a18b1668","message":"connect ECONNREFUSED 127.0.0.1:3000"}
{"time":"2026-10-18T17:06:37.553Z","requestId":"b8ea826b-21ea-45ac-8433-
    d989a18b1668","method":"GET","url":"/api/plantas","status":502,"ms":21.325,"user":"admin","ip":"127.0.0.1"}
```

4.5 Capturas en Docker, Swagger y Postman

Captura pendiente: docker-compose ps — los tres servicios en estado healthy
Guarda la imagen como `docs/capturas/docker-ps.png` y vuelve a ejecutar `python docs/generar_informe.py`.

Captura pendiente: docker-compose logs — arranque y peticiones en gateway y microservicio
Guarda la imagen como `docs/capturas/docker-logs.png` y vuelve a ejecutar `python docs/generar_informe.py`.

Captura pendiente: Swagger UI — petición exitosa a través del gateway
Guarda la imagen como `docs/capturas/swagger.png` y vuelve a ejecutar `python docs/generar_informe.py`.

Captura pendiente: Postman — ejecución exitosa de la colección
Guarda la imagen como `docs/capturas/postman.png` y vuelve a ejecutar `python docs/generar_informe.py`.

Conclusión

La arquitectura cumple los objetivos: un gateway que centraliza el acceso con cuatro mecanismos transversales (CORS, logging con trazabilidad, rate limiting y JWT), un microservicio NestJS con CRUD completo y patrones Repository/DTO/DI sobre PostgreSQL, y una orquestación reproducible con un solo comando (`docker-compose up`).